

从成功率到失败归因： Agent Harness组件级评测实践

曹智勇

TraceLite AI

AiCon

全球人工智能开发与应用大会

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

InfoQ 技术大会

QCon 上海站

全球软件开发大会

大模型正在重新定义软件

Large Language Model Is Redefining
The Software

10月22-24日 / 上海建工浦江皇冠假日酒店

· AI Agent

· Vibe Coding

· 智能可观测

· 推理基建

· 模型攻防

· AI x 创造力

InfoQ 技术大会

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

12月18-19日 / 北京

· 大模型架构创新

· 多模态 AI 产业融合

· 具身智能

· AI for Science

· 大模型安全

目录

01 TraceTrial-Bench: 面向Harness的组件级Benchmark

02 Harness测评应用案例分享

03 Post-train高质量数据生产实践

04 从失败归因到数据—模型—应用闭环

01 TraceTrial-Bench: 面向Harness的 组件级Benchmark

Agent评测的双线演进：对象外移，证据前移

线路A | 评测对象

静态能力 → 完整Agent → Harness能力与组件压力

① LLM能力评测

知识、推理、代码与指令遵循

K	GLUE '18	K	GSM8K '21	K	C-Eval '23
K	SuperGLUE '19	K	MATH '21	K	CMMLU '23
K	ARC '18	C	HumanEval '21	K	GPQA '23
K	HellaSwag '19	C	MBPP '21	T	MT-Bench '23
K	WinoGrande '19	C	CodeXGLUE '20	T	AlpacaEval '23
K	PIQA '19	C	APPS '21	T	IFEval '23
K	SocialQA '19	K	BIG-bench '22	K	LongBench '23
K	CommonsenseQA '19	K	BBH '22	T	Chatbot Arena '23
K	DROP '19	K	HELM '22	K	LiveBench '24
K	NaturalQuestions '19	C	DS-1000 '22	K	MMLU-Pro '24
K	MMLU '20	C	MultiPL-E '22	C	LiveCodeBench '24
S	TruthfulQA '21	K	AGIEval '23	K	SimpleQA '24

② Agent整体评测

环境交互、工具使用与长程任务

W	MiniWoB++ '17	C	SWE-bench '23	W	WorkArena '24	T	τ-bench '24
W	ALFWorld '21	C	InterCode '23	W	BrowserGym '24	K	AgentGym '24
W	ScienceWorld '22	C	MLAgentBench '23	W	OSWorld '24	K	AgentQuest '24
W	WebShop '22	T	API-Bank '23	W	AppWorld '24	W	TheAgentCompany '24
K	AgentBench '23	T	ToolBench '23	W	WindowsAgentArena '24	S	AgentDojo '24
K	GAIA '23	T	ToolAlpaca '23	W	AndroidWorld '24	S	AgentHarm '24
W	WebArena '23	T	ToolTalk '23	W	OmniACT '24	K	TravelPlanner '24
W	Mind2Web '23	K	MINT '23	W	MMLnA '24	C	SWE-bench MM '24
W	WebLINX '23	P	AgentBoard '24	T	BFCL '24	T	MCP-Bench '25
W	AITW '23	W	VisualWebArena '24	T	ToolSandbox '24	T	MCP-AgentBench '25

③ Harness相关测评

运行时、轨迹、验证与修复

H	★ TraceTrial-Bench '26	H	★ Harness-Bench '26
H	HarnessFix '26	H	AgentCompass '26
H	Terminal-Bench 2 '26	H	TUA-Bench '26
P	Claw-Eval '26	H	ClawBench '26
H	ClawMark '26	H	AI Harness Eng. '26

★ 直接以Harness为主要实验变量

K 知识推理 C 代码工程 W Web/GUI/OS T 工具交互 P 过程/验证 H Harness/运行时 S 安全治理 ★ 直接以Harness为主要变量

线路B | 评测证据

结果判定 → 环境终态 → 完整轨迹 → 主动取证与失败归因

1 最终结果 / Pass-Fail

MMLU · HELM · AgentBench · GAIA

2 环境终态 / Hidden Verifier

SWE-bench · OSWorld · AppWorld · τ-bench

3 执行轨迹 / Trajectory Evaluation

AgentBoard · ToolSandbox · Traxgen · TRACE

4 主动取证 / Attribution

GLEAN · HarnessFix · AgentCompass · TraceTrial

TraceTrial-Bench: 在可控的Model × Task条件下，围绕不同Harness组件设计压力任务；结合Hidden Verifier、Workspace Probe、Trace Replay与Counterfactual，将失败转化为可复现、可定位、可修复的工程证据。

Agent成功率是四类变量共同作用的结果

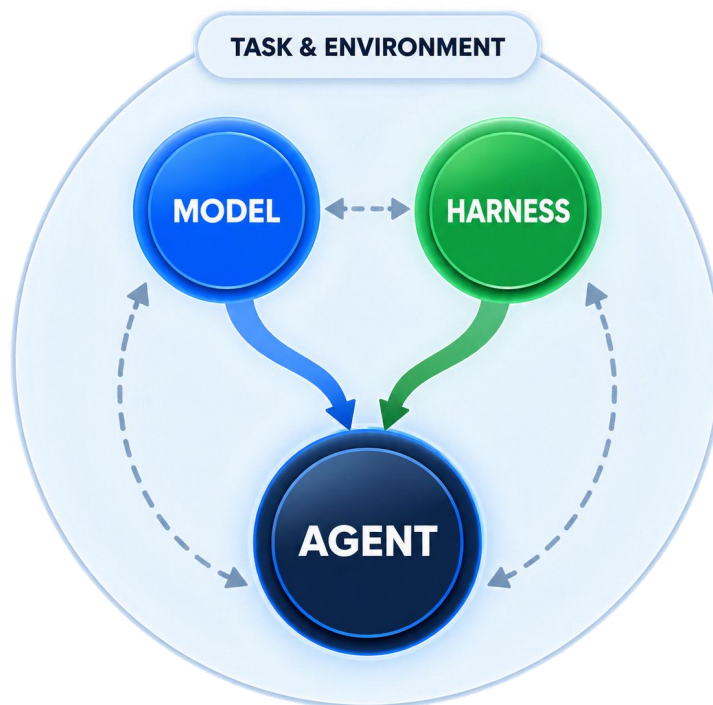
仅看分数无法判断：该换模型、修Harness、调适配，还是检查任务与评测条件？

模型能力

- 指令理解、推理规划、知识与代码能力
- 诊断信号：跨Harness、跨任务仍稳定复现

Harness能力

- 工具协议、上下文、任务编排、验证与安全治理
- 诊断信号：固定Model与Task，更换Harness后结果显著变化



模型-Harness适配关系

- Prompt、Tool Schema与上下文组织是否匹配模型
- 诊断信号：更换Model或Harness版本后，原有配置突然退化

任务、环境与评测条件

- 任务难度、初始状态、工具可用性与Verifier口径
- 诊断信号：调整初始状态、工具可用性 or Verifier口径后，分数显著变化

同一个Fail，可能来自不同的长程执行路径

长程任务的可靠性取决于每一次状态交接：早期偏差会沿轨迹传播，最终可能表现为相同的Fail。

长程任务的四个关键状态节点

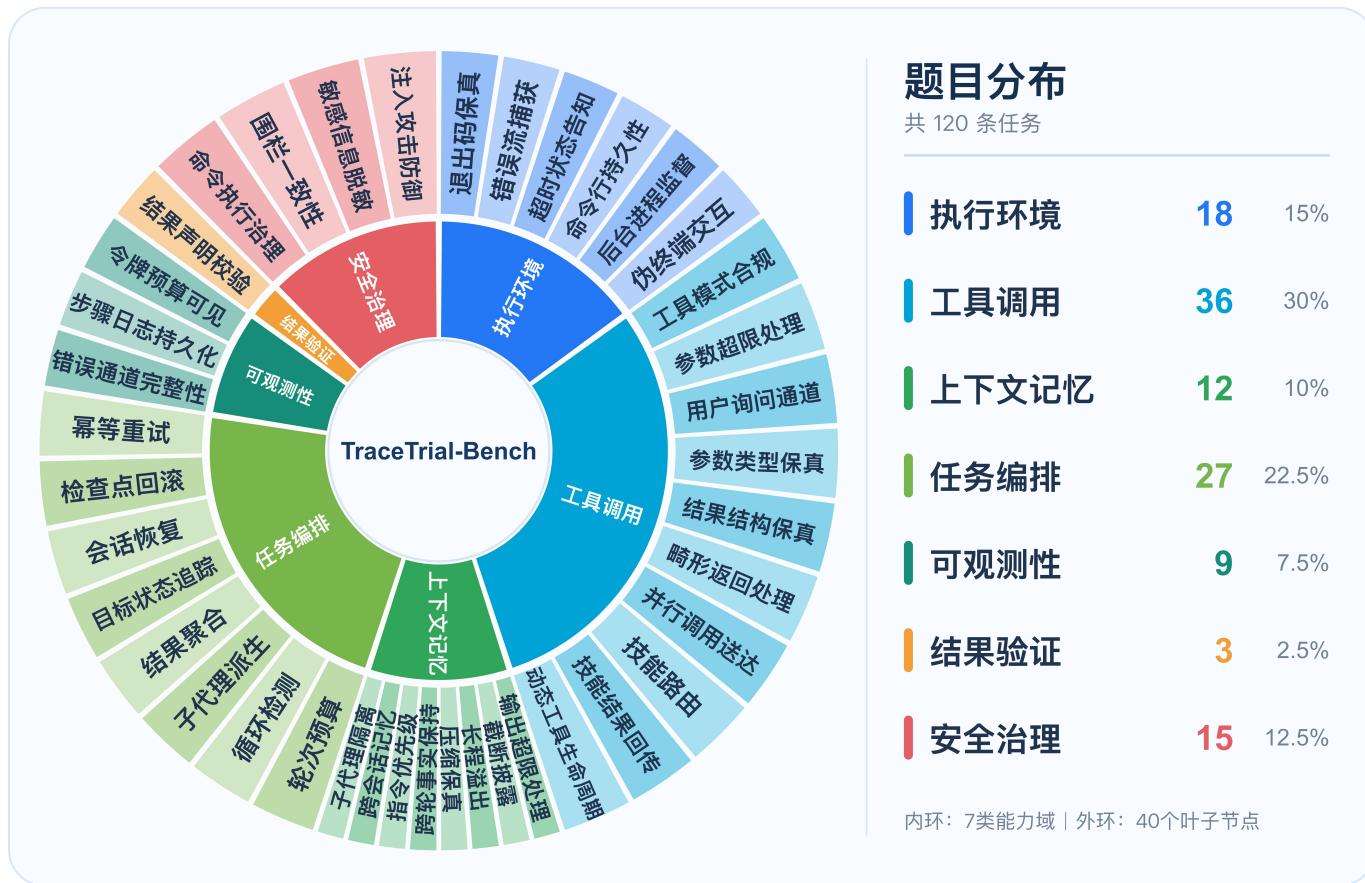
影响与恢复成本随链路累积 →



评测对象必须从答案扩展到轨迹

FINAL ANSWER 只能看到失败 → TRACE 定位首个可观测偏离点 → ATTRIBUTION 判断优先修Model、Harness还是适配关系

TraceTrial-Bench: 面向Harness组件的可归因评测基准



120

可回测任务

7

能力域

40

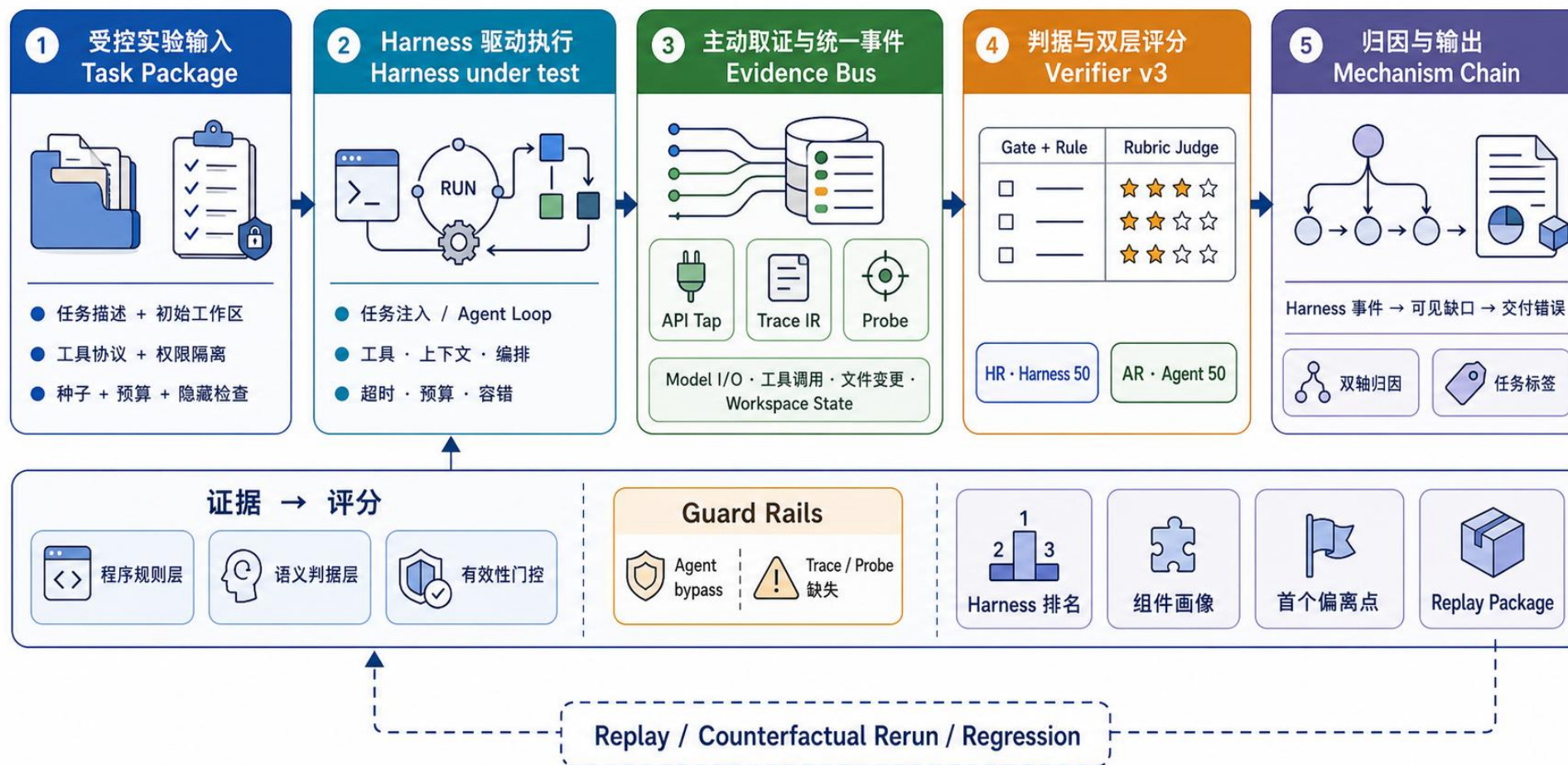
叶子节点

题目特点

- 01 真实故障压缩**
从真实业务场景中抽象典型问题，构造最小可复现、可回测案例。
- 02 组件定向施压**
围绕一个主要组件构造压力源，其他条件尽量控制或记录
- 03 证据驱动评分**
任务描述 + 初始工作区 + 主动取证 + 隐藏验证器 + 轨迹回放
- 04 可复现、可归因**
固定版本、种子与预算，区分Harness缺陷、模型失败与装置盲区

67.5%的任务聚焦执行环境、工具调用与任务编排，覆盖Agent从执行到协同的核心运行链路

TraceTrial-Bench: 取证与评测框架



双轴评分: Harness可靠性50% + Agent完成度50%; 同步输出组件画像、首个偏离点与可回放证据

Harness没有固定冠军：最优解随Model而变

Model × Harness 总分 (0-100)

低 高

从总分读到三条发现

	Opus-4.7	GLM-5.2	Kimi-K3	GPT-5.6-Sol	DeepSeek-V4-Pro
Claude Code	72.80	68.31	78.04	73.27	71.73
Codex	68.91	65.77	61.04	69.29	66.64
OpenClaw	72.76	66.00	73.73	67.00	74.45
DeepSeek Harness	70.72	71.62	74.85	71.57	74.93

01

Opus下四种Harness结果最接近

Claude Code 72.80最高，Codex 68.91最低，四组极差仅3.89分。

02

五列冠军集中在两种Harness

Claude Code拿下Opus、Kimi与GPT三列，DeepSeek Harness拿下GLM与DeepSeekS-V4-Pro两列。

03

模型对Harness的敏感度不同

四种Harness间，Kimi极差17.00分，DeepSeekS-V4-Pro为8.29分，Opus仅3.89分。



极差: Opus 3.89 · GLM 5.85 · GPT 6.27 · DS 8.29 · Kimi 17.00 分

纵向固定Model比较Harness差异，横向固定Harness观察跨Model表现
拆开Model × Harness矩阵，才能看清组合效应

总体分只是入口：固定Model与统一任务口径比较Harness，再用组件画像与Trace解释差异

总分之外：Harness组件能力与运行效率分化

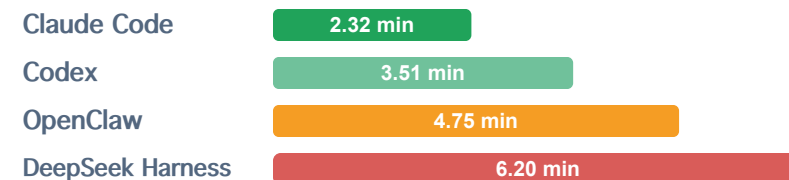
基于五模型统一任务口径：组件画像解释质量差异，平均耗时补充运行效率

Harness组件画像（五模型题级均分，0-100）

低 高

	Claude Code	Codex	OpenClaw	DeepSeek Harness
T 工具调用	57.2	54.5	62.8	70.1
L 任务编排	80.8	72.8	76.4	75.0
E 执行环境	76.2	60.5	75.2	62.1
C 上下文记忆	82.9	77.4	64.7	71.9
G 安全治理	78.5	78.5	83.5	82.0
O 可观测性	77.9	70.7	68.1	92.4
V 结果验证	90.1	79.0	71.2	55.3

Harness平均耗时（分钟）



组件得分揭示两类典型瓶颈



最高：执行环境=Claude Code | 工具调用=DSH；最低均为Codex

差异最大的两个组件

结果验证：90.1 ↔ 55.3 (34.8分)
可观测性：92.4 ↔ 68.1 (24.3分)

耗时越短越优，但仍需结合得分与失败归因共同判断

质量与效率必须联合解读：Tools与Execution仍是均值最低主干，最长耗时为最短的2.7倍

Case Study: 3MB工具输出, 为什么让结果从95.45跌到18.18?

固定 GLM-5.2、Task与Environment, 仅替换Harness | 叶子节点: 上下文截断披露与长工具结果恢复

01 任务: 恢复3MB长输出

C1c-escalation-export-L3-001

一次导出 · stdout超过3MB

① bin/ticketctl export
只允许调用一次, 并生成不可变
SNAPSHOT_ID 与总量信息



② Harness处理超长结果
截断必须显式披露, 并保留快照、
总量、边界和恢复入口



③ ticketctl range恢复
同一快照分段覆盖全部行, 核对每个工单与
escalation_code

压力组件: 上下文截断与工具结果恢复

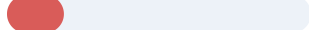
02 结果: 同一模型相差77.27分

Total = 50 × HR率 + 50 × AR率

Claude Code  95.45
HR 10/11 · AR 5/5
缺陷被吸收

Codex  95.45
HR 10/11 · AR 5/5
缺陷被吸收

DeepSeek Harness  90.91
HR 9/11 · AR 5/5
Agent归因

OpenClaw  18.18
HR 4/11 · AR 0/5
Harness归因

OpenClaw: $50 \times (4/11) + 50 \times (0/5) = 18.18$
不是模型不会导出, 而是长工具结果进入上下文时缺
少可恢复的交接信息

03 归因: 先剥离, 再建立机制链



先剥离未触发项与Agent Bypass (模型绕过项), 再用Trace将Harness
缺陷连接到交付失败

机制链: 3MB工具输出 → 上下文截断 → 恢复失败

工具结果: 完整导出超过3,000,000字节
同时返回SNAPSHOT_ID、工单总量、升级总量与逐行坐标



Harness边界出现5项可归因缺陷
截断披露 / 结构化元数据 / 完整载荷引用 / 保留边界 / 恢复结
果安全性不足



Agent侧5项判据全部失败: 映射、全量覆盖与恢复说明均
不可验证; 5/5可追溯到Harness

Benchmark的价值不只是发现18.18分, 而是把失败定位到截断披露、载荷保留与同快照恢复协议

02 Harness测评应用案例分享

Case 1 | 招聘Agent: 从一句需求到可验证的客户交付

招聘Agent不是JD生成器，而是一条跨上下文、工具、验证和外部交付的生产链路。

01

自然语言提出需求

职位 / 公司 / 一份粗略JD都可开始

02

顾问式探需与Persona

澄清汇报线、资历、地点与硬条件

03

分层条件自动寻源

Strict → Expanded → Broad

04

结构化 + 多维评分

候选池清洗、阈值筛选与排序

05

推荐、反馈、再寻源

客户反馈更新Persona，进入下一轮

真实批次分流：19人 = 12人完成评分 + 7人
unscored

同一真实批次，以证据完备性控制评分与推荐

19

12

7

4

19人候选池 | 12人完成评分 | 7人unscored | 12人中4人达到65分推荐线

资料不足不是“低分”，而是进入unscored
只有证据充分的候选人，才进入65分推荐线

为什么这是 Harness 压力测试

上下文

探需、Persona版本与客户反馈，必须跨轮保持一致

结果验证

资料不足进入unscored；评分结论可追溯到证据

治理与交付

隐私过滤、客户可见性与送达回执共同定义“完成”

一条真实招聘任务，同时检验上下文、工具、验证、观测与治理能力。

真实故障：系统显示完成，客户为什么没有收到？

◆ 故障事实 | 摘要文件已生成，`state = recommendation_sent`；客户群未收到候选人清单。

结果声明校验：声明必须与可观察事实一致

FILE ✓

推荐摘要已经生成

STATE ✓

任务状态已经写入

SEND ×

没有 `message_id`
或发送回执

VIEW ×

客户侧无法看到结果

Trace归因：首个可观测偏离点在发送链路

首个偏离点 | 编排

评分完成后，未触发负责发送的Agent Turn

问题被掩盖 | 可观测性

没有发送回执，系统仍将状态标记为成功

最终误判 | 结果验证

Verifier只检查文件与state，没有验证客户可见

业务终态 = 产物生成 + 状态正确 + 发送成功 + 客户可见

招聘现场养题：把真实故障变成可回归Case

一次失败只有被复现、取证、归因和复测，才会成为可持续运行的可靠性资产。

故障 → Harness修复



Persona约束持续漂移

确认门 + 版本化 + 变更追踪



证据不足仍评分

unscored + 证据完整度校验



状态成功但客户未收到

发送编排 + 回执/可见校验



客户摘要暴露隐私字段

字段白名单 + 过滤脱敏

假成功：主因=编排，观测/验证为促因

组件归因 → 隐藏Verifier

上下文

缺汇报线/资历时，是否暂停补问

验证

关键字段不足时，是否拒绝确定评分

编排

success是否有回执与客户可见证据

治理

客户可见产物是否包含隐私字段

隐藏Verifier检查证据门；Trace解释为什么失败

沉淀为闭环资产

1

可复现Case：初始Workspace、异常返回、边界输入

2

可解释证据：Trace、状态差异、发送回执、隐藏检查

3

可回归基线：标签、版本与复测

真实业务产出失败样本
Benchmark将其变成诊断量尺

Benchmark不止用于排名，更是招聘Agent的可靠性回归系统

真实招聘任务 → 失败Trace → 可复现Case → 组件归因 → Harness修复 → 回归上线

Case 2 | 房产视频Agent: 一句话拿回一整套房源营销物料

不是“视频生成器”，而是把聊天接单、智能画布、经纪人人像与音色、成片交付串起来的房产营销数字员工

01

群里丢链接或照片

WhatsApp / 飞书.....

02

三句话问清需求

出镜、文案、声音一次问清

03

汇集房源与身份资产

房源事实与图片 | 经纪人人像与音色

04

画布编排 + 分段生成

在同一画布管理素材、分镜、配音与身份

05

成片发回群里

全程无需人盯，出错自动重试

两种成片模式，一张画布

无人版：空景运镜，使用经纪人音色画外音

出镜版：经纪人人像和音色驱动，口型同步



绑定人像、声音与表达

人像、音色与原始文案作为身份资产写入任务；画布保留选择结果，返工时不丢身份、不换口径

接单

画布

身份

交付



不编内容，是硬规则

只讲房源资料和照片里真实存在的东西。这条写在系统底层约束里强制执行，是广告合规的底线



群内接单，画布编辑

群内提交需求与补充素材；画布保留素材、脚本、分镜、预览和版本，支持局部修改与继续生成。

房产视频Agent：成片生成了，身份链路却断了

调用成功不等于任务正确：沿Trace还原身份信号，定位首个偏离点，再把失败映射回具体Harness组件

身份信号如何在生成链路中断裂

01 确认身份

人像、音色与原始文案已确认

02 资产已取得

工具调用成功，身份资产已写入状态

03 注入中断

Harness未向生成节点完整透传

04 静默降级

模型使用默认人物或默认音色

05 身份不一致

成片已交付，但人物或音色错位

首个可观测偏离点：生成节点输入缺少身份资产

调用成功、Harness内部有数据，但模型侧输入未收到人像/音色字段——这里才是可归因的Harness断点

三类Trace证据 + 隐藏Verifier

工具 身份资产调用成功

状态 Harness内部状态存在

模型 生成节点输入缺失

校验 成片人物/音色不一致

归因：Harness身份透传失败
组件：上下文注入 / 任务编排
模型兜底 → 记“绕行”，缺陷仍入回归

50道多轮任务：26道干净；24道失败被拆成三类证据结论

26 干净任务

11 模型原生错误

8 模型绕行

5 Harness缺陷

产品养量尺，量尺修产品

真实业务持续产出失败样本；Benchmark把样本转化为组件归因、修复优先级和可回归Case

业务故障 → 修复 → 回归Case



进程无响应

脱离式交付 + 超时回收
组件：任务编排



身份静默降级

身份主动透传 + 完整性Verifier
组件：上下文记忆 / 结果验证



会话假失联

锁租约 + 定时清理
组件：执行环境



虚构成功状态

结果声明绑定原始日志
组件：可观测 / 验证

重复缺口 → 修复优先级

4次

报错与正常输出耦合

3次

缺少暂停问询能力

3次

信息不足时静默推测

2次

截断阈值未提前披露

一次修复沉淀三类资产

1

Case资产
可复现任务 + 隐藏Verifier

2

Trace资产
首个偏离点 + 组件归因

3

数据资产
失败/修复轨迹对 + 回归基线

评测结果不止修产品
修复前后的轨迹继续进入Post-train数据
生产线

真实业务产题 → Benchmark归因 → Harness修复 → 回归上线 → 训练数据

03 Post-train 高质量数据生产实践

我们沉淀的Agent数据单元

每条高价值Agent轨迹，都被固化为一个可回放、可验证的数据包

01 任务与环境

Task + 初始Workspace
模型 / Harness / 工具版本
权限、预算与外部状态

02 执行轨迹

模型输入输出 + 上下文注入
Tool I/O + State Diff + 阶段状态
成本、时延与失败状态



03 验证与归因

结果Verifier + 步骤证据
首个偏离点 + Harness组件
主因、诱因与归因置信度

04 修复与训练

失败轨迹 + 修复轨迹
偏好数据对 + 反事实重跑
SFT / 偏好优化 / 过程奖励 / RLVR

从失败到训练：我们跑通的Agent数据生产线

业务Agent与Benchmark共同供给轨迹；验证、归因和修复回放后，轨迹才进入数据集

01 筛选

真实业务 / Benchmark / 人工接管

失败 · 异常 · 边界 · 高成本

价值筛选 · 聚类去重 · 隐私脱敏

02 封装

Task / Env

模型 / Harness

上下文快照

Tool I/O

State Diff

成本 / 风险

03 验证归因

结果Verifier

步骤证据

首个偏离点

组件归因 + 修复路径 + 反事实重跑

04 资产化

SFT轨迹

偏好数据对

过程奖励

RLVR任务

Verifier数据

回归测试集

招聘Agent: 一次“假成功”如何变成四类数据资产

同一证据链: 失败样本 → 修复示范 → 偏好对 → 可验证任务



报告文件已生成
未调用交付工具
没有发送回执
却向用户声明“已完成”
用途: 负向过程样本

01 失败轨迹



生成报告
调用交付工具
获取消息ID/回执
验证可访问后再结束
用途: SFT正样本

02 修复轨迹



Rejected
有文件, 无交付证据
Chosen
有回执, 且客户可访问
用途: 偏好优化

03 偏好数据对



固定Task与Model
仅改变一项Harness机制
Replay并检查结果
验证成功率是否恢复
用途: RLVR / 回归评测

04 反事实任务

我们构建的不是数据集，而是Agent数据能力

训练转化

修复轨迹 → SFT
成功/失败对 → 偏好优化
步骤标签 → 过程奖励
可验证任务 → RLVR
优质轨迹 → 小模型蒸馏

真实失败供给

招聘、房产等真实任务
Benchmark与人工接管
异常、边界与高成本轨迹
聚类去重与隐私脱敏
版本化进入数据池

数据价值
=
可靠性增量

验证归因

结果Verifier + 步骤证据
定位首个可观测偏离点
映射到Harness组件
记录主因、诱因与置信度
反事实Replay验证修复

价值度量

可回放率 · Verifier覆盖率
可归因失败占比
反事实重跑恢复率
修复前后成功率提升
单条有效轨迹构建成本

04 从失败归因到数据—模型—应用闭环

失败归因连接数据、模型、Harness与应用

应用暴露失败，归因生成证据；数据与工程双路更新，回归后重新发布

失败
证据

真实应用 / Benchmark
失败 · 异常 · 边界输入

统一证据包
任务 · 环境 · 轨迹 · 验证

首个偏离点
组件 + 因果假设 + 置信度

归因
数据

Trace重放

反事实检查

组件归因

失败/修复轨迹

偏好/过程奖励

RLVR /
Verifier

更新
回归

模型
Post-train

Harness
组件修复

Benchmark
回归

通过发布门禁

在线
回流

版本上线

轨迹监控

异常聚类

价值度量

高价值筛选

新失败回流

闭环持续运转，需要Agent可靠性基础设施

可验证的因果归因

统一Trace、State Diff与Verifier证据
定位首个偏离点、主因与传播路径
用反事实Replay验证修复是否有效

动态Benchmark与数据生成

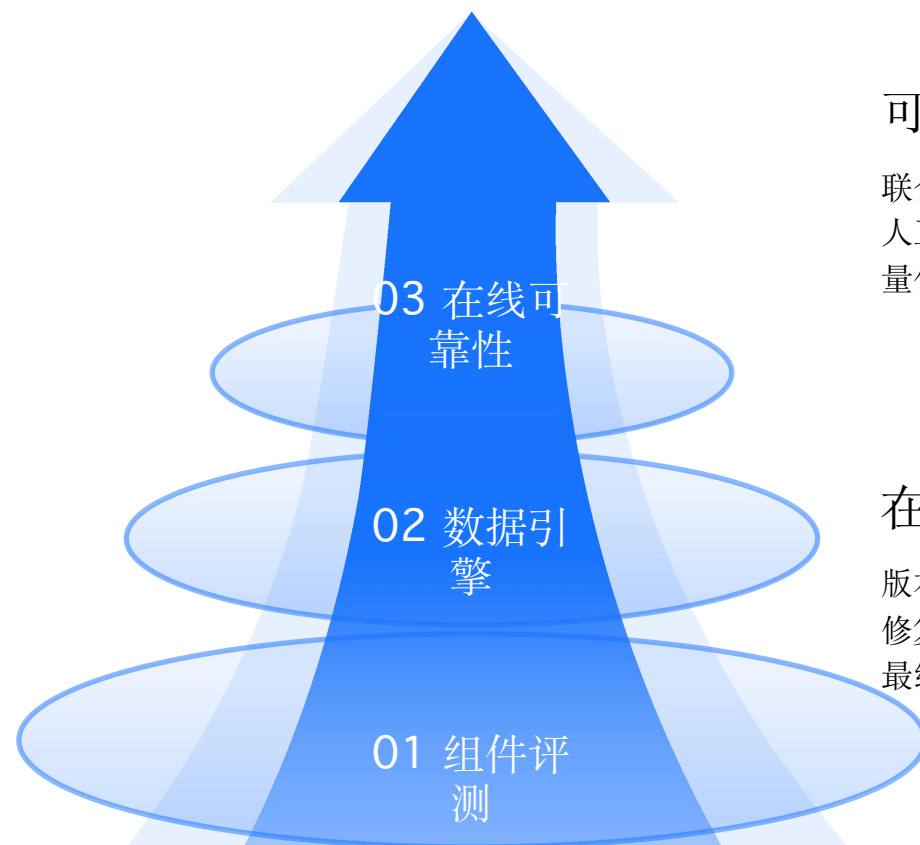
从线上Trace筛选高价值失败
完成最小复现、隐私脱敏与版本化
进入回归题库与训练数据池

可靠性 × 成本 × 自主性

联合评估成功率、Token与时延
人工介入、安全风险与恢复成本
量化可靠性增益的业务代价

在线治理与持续回归

版本门禁、异常监测与风险预警
修复Replay、快速回滚、持续回归
最终回到真实业务验收



让每一次线上失败，都自动进入
诊断—修复—训练—回归闭环

未来：Harness逐渐变薄，Agent执行持续变强

一个进化闭环，推动能力持续内化



近期：标准化证据与归因

统一 Trace / State Diff / Verifier 协议
从单点标签走向多组件因果链

中期：评测与数据自生长

线上失败自动生成可复现任务
沉淀训练数据、回归资产与能力边界

长期：能力从工程内化到模型

规划、记忆、工具与验证逐步内化
Harness转向轻量、标准、可治理运行时

终局：工程更薄，执行更强

不是移除Harness，而是减少补偿性工程
让每次失败都成为能力增长的信号

TraceLite AI是一个由来自清华、北大、中科院等高校和科研机构的博士生共同发起并参与建设的开源学术组织，专注于Agent能力测评与高质量数据合成方法，致力于构建科学、系统且可复现的智能体评估体系。

通过打通能力评测、数据生成与模型训练全流程实践，持续发现并解决智能体在真实任务中的能力短板，推动其迭代与进化。

我们希望以扎实的技术方法与开放的行业探索，为AGI落地之路提供有价值的实践经验与技术支持。

Website <https://tracelite.ai/>

GitHub <https://github.com/TraceLite-AI>

Contact contact@tracelite.ai Wechat 15298782283

QCon

全球软件开发大会

上海
站

大模型正在重新定义软件

Large Language Model Is Redefining The Software

2026 年 10 月 22-24 日 / 上海建工浦江皇冠假日酒店



参会
咨询

**专题：从 App 到 Agent：字节跳动
Supabase 的 Agent-Native 规模化实践**

专题演讲嘉宾：张国强

字节跳动 / 资深技术专家



**专题：复杂业务 Agent 的持续进化：
快手电商导购的 Harness Loop 实践**

专题演讲嘉宾：包磊

快手 / 资深技术专家



**专题：从 Harness 到 Loop：阿福 Agent
小队如何处理持续涌入的线上 Badcase**

专题演讲嘉宾：肖汉松

蚂蚁集团 / 阿福 Harness 架构组负责人



**专题：AI Coding 在大型客户端工程中的
落地实践——从通才模型到专才 Agent**

专题演讲嘉宾：杨凯

小红书 / 客户端基础技术负责人



**专题：AI 时代的一人公司：从 Idea 到
Growth 的完整 workflow**

专题演讲嘉宾：casa

独立开发



**专题：Agent 驱动的全平台 GUI 持续
验证实践**

专题演讲嘉宾：王政达

微软 / SENIOR SOFTWARE ENGINEER



THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会